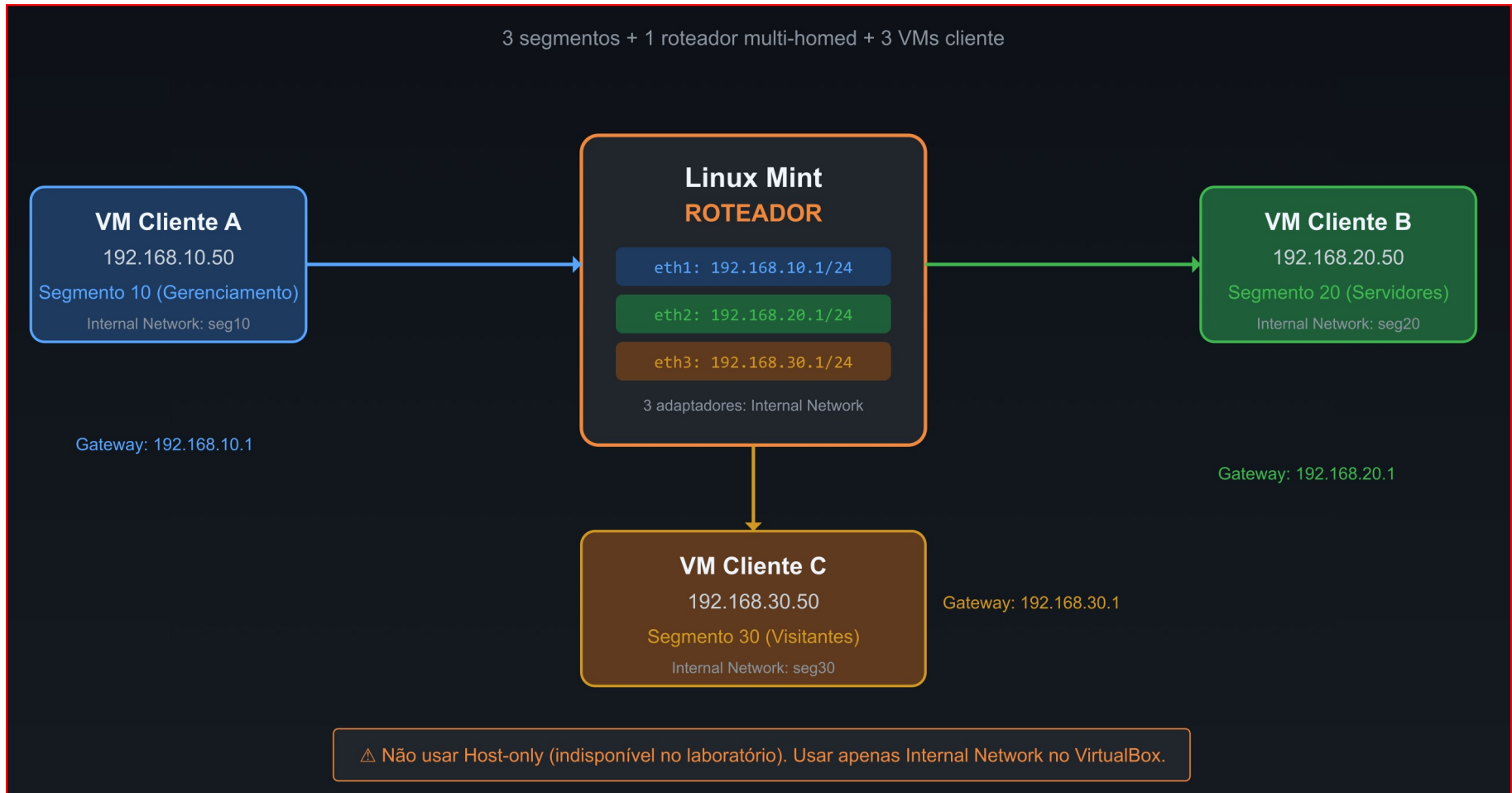


**Segurança em Sistemas Operacionais e Redes de
Computadores II**
ISG012-02 - Prática - Roteamento Seguro e ACLs

Objetivo

- Configurar uma VM Linux Mint como roteador multi-homed com três interfaces de rede conectadas a segmentos distintos no VirtualBox;
- Implementar roteamento estático entre os segmentos utilizando ip route e ip rule;
- Configurar nftables para filtrar tráfego entre interfaces, bloqueando comunicação entre duas VLANs simuladas e permitindo apenas tráfego SSH entre um segmento de gerenciamento e um segmento de servidores.



- Segmento 10 (Gerenciamento): Internal Network "seg10"
- Segmento 20 (Servidores): Internal Network "seg20"
- Segmento 30 (Visitantes): Internal Network "seg30"

Preparação do Ambiente VirtualBox

- Configuração das VMs:
 - VM Roteador (Linux Mint):
 - 3 adaptadores de rede em modo "Internal Network"
 - Adapter 1: seg10 (eth1)
 - Adapter 2: seg20 (eth2)
 - Adapter 3: seg30 (eth3)
 - Memória: 2GB, Disco: 20GB
 - VMs Cliente A, B, C (Linux Mint ou Windows):
 - 1 adaptador no segmento correspondente
 - Gateway padrão apontando para o IP do roteador no segmento
- Nota: Não usar "Host-only" (indisponível no laboratório). Usar apenas "Internal Network".

Configuração de Endereços IP no Roteador

- Script Bash para configuração de interfaces:

```
bash
#!/bin/bash
# setup_router.sh — Configura interfaces do roteador
# Habilitar IP forwarding
sudo sysctl -w net.ipv4.ip_forward=1
echo "net.ipv4.ip_forward=1" | sudo tee -a /etc/sysctl.conf
# Configurar eth1 (Segmento 10)
sudo ip addr add 192.168.10.1/24 dev eth1
sudo ip link set eth1 up
# Configurar eth2 (Segmento 20)
sudo ip addr add 192.168.20.1/24 dev eth2
sudo ip link set eth2 up
# Configurar eth3 (Segmento 30)
sudo ip addr add 192.168.30.1/24 dev eth3
sudo ip link set eth3 up
echo "Interfaces configuradas:"
ip addr show | grep "inet "
```

- Execução: `chmod +x setup_router.sh && ./setup_router.sh`

Configuração de Roteamento Estático

- Script Bash para rotas entre segmentos:

```
bash
```

```
#!/bin/bash
```

```
# routes_static.sh — Rotas estáticas entre segmentos locais
```

```
# As rotas para redes diretamente conectadas já existem
```

```
# Verificar tabela de roteamento:
```

```
echo "=== Tabela de Roteamento Principal ==="
```

```
ip route show
```

```
# Adicionar rota default se houver acesso externo (opcional)
```

```
# sudo ip route add default via 192.168.10.254 dev eth1
```

```
# Verificar conectividade entre segmentos
```

```
echo "=== Teste de ping para Cliente A (192.168.10.50) ==="
```

```
ping -c 3 192.168.10.50
```

```
echo "=== Teste de ping para Cliente B (192.168.20.50) ==="
```

```
ping -c 3 192.168.20.50
```

- Observação: Em redes diretamente conectadas, o Linux já possui as rotas automaticamente. O roteamento estático é necessário apenas para redes remotas.

Policy-Based Routing (PBR) com ip rule

- Script Bash para PBR — Isolamento de tráfego de visitantes:

```
bash
```

```
#!/bin/bash
```

```
# pbr_setup.sh — Policy-Based Routing para segmento 30
```

```
# Criar tabela customizada
```

```
echo "200 visitantes" | sudo tee -a /etc/iproute2/rt_tables
```

```
# Adicionar rota default específica para segmento 30 (se houver internet)
```

```
# sudo ip route add default via 192.168.30.254 dev eth3 table visitantes
```

```
# Regra: tráfego originado do segmento 30 usa tabela visitantes
```

```
sudo ip rule add from 192.168.30.0/24 table visitantes priority 100
```

```
# Verificar regras
```

```
echo "=== Regras de Roteamento ==="
```

```
ip rule show
```

```
# Verificar rotas na tabela visitantes
```

```
echo "=== Tabela visitantes ==="
```

```
ip route show table visitantes
```

Configuração Inicial do nftables

- Script Bash para estrutura base do firewall:
bash
#!/bin/bash
nftables_base.sh — Estrutura inicial do nftables
sudo tee /etc/nftables.conf << 'EOF'
#!/usr/sbin/nft -f
flush ruleset
table inet filter {
 # Definir variáveis para interfaces
 define MGMT_DEV = "eth1"
 define SERV_DEV = "eth2"
 define GUEST_DEV = "eth3"
 # Chain input — protege o próprio roteador
 chain input {
 type filter hook input priority filter; policy drop;
 # Aceitar loopback
 iifname "lo" accept
 # Aceitar conexões estabelecidas
 ct state established,related accept

...continua no próximo slide

Configuração Inicial do nftables

```
# Permitir SSH no roteador apenas do segmento de gerenciamento
iifname $MGMT_DEV tcp dport 22 accept
# Permitir ping limitado
iifname $MGMT_DEV icmp type echo-request limit rate 5/second accept
# Logar e dropar o resto
log prefix "DROP-INPUT: " drop
}
# Chain forward — controla tráfego entre segmentos
chain forward {
    type filter hook forward priority filter; policy drop;
    # Aceitar conexões estabelecidas
    ct state established,related accept
}
# Chain output — tráfego originado do roteador
chain output {
    type filter hook output priority filter; policy accept;
}
}
EOF
# Aplicar configuração
sudo nft -f /etc/nftables.conf
# Verificar regras
sudo nft list ruleset
```

ACLs Avançadas — Regras de Segmento

- Script Bash para regras de forward entre segmentos:

```
bash
```

```
#!/bin/bash
```

```
# nftables_forward.sh — Regras de ACL entre segmentos
```

```
sudo tee /etc/nftables_forward.conf << 'EOF'
```

```
#!/usr/sbin/nft -f
```

```
table inet filter {
```

```
    define MGMT_DEV = "eth1"
```

```
    define SERV_DEV = "eth2"
```

```
    define GUEST_DEV = "eth3"
```

```
    chain forward {
```

```
        type filter hook forward priority filter; policy drop;
```

```
        ct state established,related accept
```

```
# REGRA 1: Gerenciamento pode acessar Servidores em qualquer porta
```

```
        iifname $MGMT_DEV oifname $SERV_DEV accept
```

...continua na próxima página

ACLs Avançadas — Regras de Segmento

```
# REGRA 2: Gerenciamento pode acessar Visitantes (monitoramento)
iifname $MGMT_DEV oifname $GUEST_DEV accept
# REGRA 3: Visitantes NÃO podem acessar Servidores (isolation)
iifname $GUEST_DEV oifname $SERV_DEV drop
# REGRA 4: Visitantes NÃO podem acessar Gerenciamento
iifname $GUEST_DEV oifname $MGMT_DEV drop
# REGRA 5: Servidores podem responder a Visitantes (established já cobre)
# Mas novas conexões de Servidores para Visitantes são bloqueadas por policy drop
# REGRA 6: Servidores podem acessar internet (se houver) via MGMT
iifname $SERV_DEV oifname $MGMT_DEV accept
# Logar tentativas bloqueadas
log prefix "DROP-FORWARD: " drop
}
}
EOF
sudo nft -f /etc/nftables_forward.conf
sudo nft list chain inet filter forward
```

Bloqueio Específico Entre VLANs

- Script Bash para regras granulares de VLAN:

```
bash
```

```
#!/bin/bash
```

```
# vlan_block.sh — Bloqueio específico entre segmentos
```

```
sudo tee /etc/nftables_vlan.conf << 'EOF'
```

```
#!/usr/sbin/nft -f
```

```
table inet filter {
```

```
    define MGMT_DEV = "eth1"
```

```
    define SERV_DEV = "eth2"
```

```
    define GUEST_DEV = "eth3"
```

```
    chain forward {
```

```
        type filter hook forward priority filter; policy drop;
```

```
        ct state established,related accept
```

```
        # Permitir SSH de Gerenciamento para Servidores
```

```
        iifname $MGMT_DEV oifname $SERV_DEV tcp dport 22 accept
```

```
        # Permitir HTTP/HTTPS de Gerenciamento para Servidores
```

```
        iifname $MGMT_DEV oifname $SERV_DEV tcp dport { 80, 443 } accept
```

...continua no próximo slide

Bloqueio Específico Entre VLANs

```
# BLOQUEAR explicitamente tráfego entre Visitantes e Servidores
iifname $GUEST_DEV oifname $SERV_DEV log prefix "VLAN-ISOLATION: " drop
# BLOQUEAR explicitamente tráfego entre Visitantes e Gerenciamento
iifname $GUEST_DEV oifname $MGMT_DEV log prefix "VLAN-ISOLATION: " drop
# Permitir que Visitantes acessem apenas DNS externo (se houver)
# iifname $GUEST_DEV oifname $MGMT_DEV udp dport 53 accept
# Dropar todo o resto entre segmentos não autorizado
log prefix "DROP-INTER-VLAN: " drop
}
}
EOF
sudo nft -f /etc/nftables_vlan.conf
```

Testes de Conectividade

- Script Bash para validação do laboratório:

```
bash
```

```
#!/bin/bash
```

```
# test_lab.sh — Testes de validação
```

```
echo "=== Teste 1: Cliente A (Gerenciamento) -> Cliente B (Servidor) ==="
```

```
ping -c 3 192.168.20.50
```

```
echo "=== Teste 2: Cliente A -> SSH no Cliente B ==="
```

```
ssh 192.168.20.50 "echo 'SSH OK'"
```

```
echo "=== Teste 3: Cliente C (Visitante) -> Cliente B (Servidor) ==="
```

```
# Deve FALHAR (bloqueado pelo nftables)
```

```
ping -c 3 192.168.20.50
```

```
echo "=== Teste 4: Cliente C -> Cliente A (Gerenciamento) ==="
```

```
# Deve FALHAR (bloqueado pelo nftables)
```

```
ping -c 3 192.168.10.50
```

```
echo "=== Teste 5: Verificar logs de bloqueio ==="
```

```
sudo dmesg | grep "VLAN-ISOLATION" | tail -5
```

Verificação e Persistência

- Comandos para verificar e persistir a configuração:

```
bash
```

```
# Verificar regras nftables atuais
```

```
sudo nft list ruleset
```

```
# Verificar tabela de roteamento
```

```
ip route show
```

```
ip rule show
```

```
# Verificar IP forwarding
```

```
sysctl net.ipv4.ip_forward
```

```
# Persistir nftables
```

```
sudo systemctl enable nftables
```

```
sudo systemctl restart nftables
```

```
# Persistir IP forwarding
```

```
echo "net.ipv4.ip_forward=1" | sudo tee /etc/sysctl.d/99-forward.conf
```

```
sudo sysctl -p /etc/sysctl.d/99-forward.conf
```

...continua no próximo slide

Verificação e Persistência

Criar script de inicialização para rotas (exemplo com systemd)

```
sudo tee /etc/systemd/system/lab-routes.service << 'EOF'
```

```
[Unit]
```

```
Description=Lab Routing Configuration
```

```
After=network.target
```

```
[Service]
```

```
Type=oneshot
```

```
ExecStart=/home/mint/setup_router.sh
```

```
RemainAfterExit=yes
```

```
[Install]
```

```
WantedBy=multi-user.target
```

```
EOF
```

```
sudo systemctl enable lab-routes.service
```

Troubleshooting Comum

Problema	Causa Provável	Solução
Ping falha entre segmentos	IP forwarding desabilitado	<code>`sysctl -w net.ipv4.ip_forward=1`</code>
Cliente não encontra gateway	Gateway errado ou interface down	Verificar <code>`ip addr`</code> e configuração do cliente
nftables não carrega	Sintaxe incorreta	<code>`nft -c -f /etc/nftables.conf`</code> (check)
Regra não funciona	Interface nomeada incorretamente	Verificar com <code>`ip link show`</code>
Tráfego bloqueado indevidamente	Policy drop sem regra established	Adicionar <code>`ct state established,related accept`</code>

Checklist de Encerramento

- ☐ IP forwarding habilitado e persistente
- ☐ Interfaces configuradas com IPs corretos
- ☐ Rotas entre segmentos funcionando (teste ping)
- ☐ nftables com policy drop em input e forward
- ☐ SSH permitido apenas do segmento de gerenciamento
- ☐ Isolamento completo entre Visitantes e Servidores
- ☐ Logs configurados para auditoria
- ☐ Configuração persistida para reboot
- ☐ Documentação da topologia e regras salva

Arquivos da disciplina

- Link para aGoogle Drive com os arquivos da disciplina:
 - https://drive.google.com/drive/folders/1KuGQgeDWI5RLBZntXmgqlx6NkJnvQx_i?usp=sharing
- Link encurtado:
 - <https://ziply.pk/QbJHLB>